

ECE 310

Lab 2 Report

Visnu Muthuraman

February 10, 2026

Abstract

In this lab, a full adder was built using basic logic gates and then used to create a 4-bit ripple-carry adder (RCA). The design was written using structural modeling in Verilog. The goal was to understand how carry signals move through each stage of the adder. A testbench was used to apply several input cases, including zero inputs, overflow cases, and full carry propagation. The simulation results showed that the adder produced the correct sum and carry outputs for all test cases.

1 Introduction, Background, and Theory

A full adder is a circuit that adds two one-bit binary numbers and a carry-in. It produces a sum and a carry-out. A full adder can be built using XOR, AND, and OR gates. The XOR gates are used to compute the sum, and the AND and OR gates are used to compute the carry-out.

When several full adders are connected together, a ripple-carry adder is formed. In this design, the carry-out from one stage becomes the carry-in of the next stage. A 4-bit ripple-carry adder uses four full adders connected in series.

One important feature of this design is carry delay. Each stage must wait for the previous carry before producing its final result. This makes ripple-carry adders simple but slower for larger bit sizes.

2 Design

The design was completed in two parts. First, a gate-level full adder was created. Then, four full adders were connected to build the 4-bit ripple-carry adder.

2.1 Full Adder

The full adder was implemented using basic logic gates as shown below.

```
1 module full_adder_struct(  
2     output S, Cout,  
3     input A, B, Cin  
4 );  
5  
6 wire xor1, and2, and3;  
7  
8 xor g1(xor1, A, B);  
9 and g2(and2, A, B);  
10 xor g3(S, xor1, Cin);  
11 and g4(and3, xor1, Cin);  
12 or g5(Cout, and3, and2);  
13  
14 endmodule
```

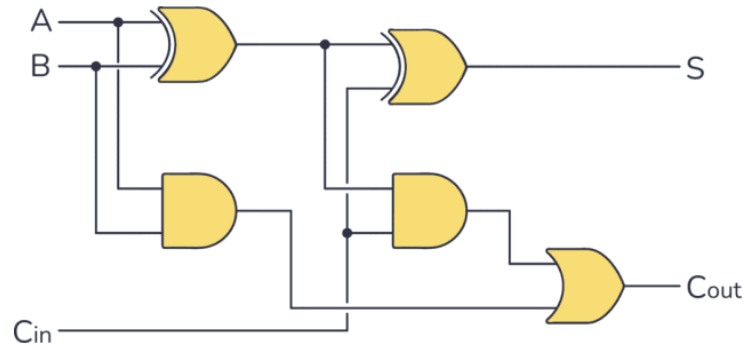


Figure 1: Full Adder Diagram (Gate-Level).

2.2 4-Bit Ripple-Carry Adder (RCA)

Four full adders were connected in series to build the 4-bit ripple-carry adder.

```

1 module rca_4bit(
2     output [3:0] S,      // Sum
3     output Cout,        // Carry Out
4     input  [3:0] A, B    // A & B
5 );
6
7 wire cin1, cin2, cin3;
8
9 full_adder_struct f0(S[0], cin1, A[0], B[0], 1'b0);
10 full_adder_struct f1(S[1], cin2, A[1], B[1], cin1);
11 full_adder_struct f2(S[2], cin3, A[2], B[2], cin2);
12 full_adder_struct f3(S[3], Cout, A[3], B[3], cin3);
13
14 endmodule

```

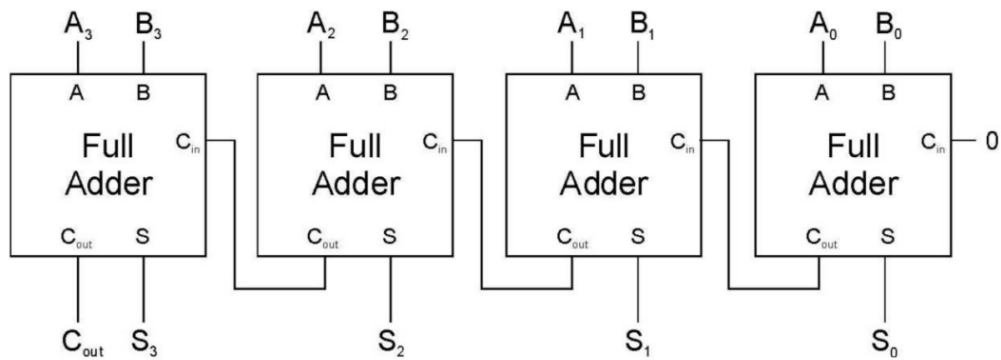


Figure 2: 4-bit Ripple-Carry Adder Diagram (Structural).

2.3 Testbench

A testbench was written to check the behavior of the adder.

```

1 module rca_4bit_tb;
2     reg [3:0] A, B;      // Inputs to the adder
3     wire [3:0] S;       // Sum output
4     wire Cout;          // Carry-out
5

```

```

6   rca_4bit dut (S, Cout, A, B);
7
8   initial begin
9       A = 4'b0000;
10      B = 4'b0000;
11      #10;
12      $display("A: %b B: %b S: %b Cout: %b ", A, B, S, Cout);
13
14      A = 4'b1111;
15      B = 4'b1111;
16      #10;
17      $display("A: %b B: %b S: %b Cout: %b ", A, B, S, Cout);
18
19      A = 4'b0100;
20      B = 4'b0011;
21      #10;
22      $display("A: %b B: %b S: %b Cout: %b ", A, B, S, Cout);
23
24      A = 4'b1111;
25      B = 4'b0001;
26      #10;
27      $display("A: %b B: %b S: %b Cout: %b ", A, B, S, Cout);
28   end
29 endmodule

```

These test cases were chosen to check normal operation, overflow, and carry propagation.

3 Results

The simulation results were observed using waveform output. The following cases were tested:

- **Test 1: 0000 + 0000:** Tests addition of two zeros (base-case); ensures sum and carry-out are correct.
- **Test 2: 1111 + 1111:** Tests for overflow; verifies that carry-out (**Cout**) is generated correctly.
- **Test 3: 0100 + 0011:** Tests normal addition with no overflow; checks correct sum calculation.
- **Test 4: 1111 + 0001:** Tests addition causing 4-bit overflow; ensures carry propagates to **Cout**.

All outputs matched the expected binary results. Carry signals moved from the least significant bit to the most significant bit when required. No errors were observed during simulation.

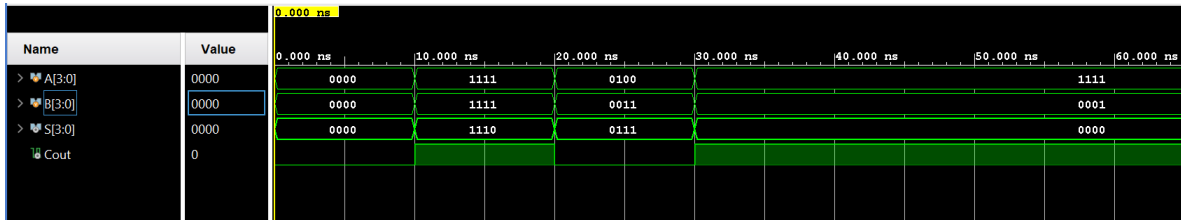


Figure 3: Simulation waveform of the 4-bit ripple-carry adder showing sum and carry outputs for all test cases.

4 Conclusion

A 4-bit ripple-carry adder was successfully implemented using gate-level full adders in Verilog. The design was tested using several input combinations, and correct results were observed in all cases. The

lab demonstrated how carry signals propagate in a ripple-carry structure and reinforced basic digital logic design concepts.

This report was compiled using L^AT_EX .